

# ***Manual Técnico Módulo 4***



Os Fundos Europeus mais próximos de si.



Cofinanciado pela  
União Europeia

|                 |
|-----------------|
| <b>Módulo 4</b> |
|-----------------|

|                                                 |          |
|-------------------------------------------------|----------|
| <b>1. Tipos de Estruturas de Dados .....</b>    | <b>3</b> |
| 1.1. Função .....                               | 3        |
| 1.2. Parâmetros Imutáveis vs. Mutáveis .....    | 4        |
| 1.3. Parâmetros por Defeito .....               | 4        |
| 1.4. Variáveis Globais vs Locais .....          | 5        |
| 1.5. Isolamento de Input .....                  | 5        |
| 1.6. Função main .....                          | 6        |
| 1.7. Controlo de Exceções .....                 | 7        |
| 1.8. Gestão de Recursos e Código de Saída ..... | 8        |

## 1. Tipos de Estruturas de Dados

A transição para a arquitetura funcional representa um marco estrutural no desenvolvimento. Em cibersegurança, a criação de ferramentas através de *scripts* monolíticos — executados sequencialmente de cima para baixo — resulta em código difícil de auditar, complexo de isolar para testes e propenso a anomalias operacionais. A solução técnica para estes desafios exige a adoção da modularização e da engenharia de software defensivo.

### 1.1. Função

Uma função constitui um bloco de código autónomo desenhado para executar uma tarefa específica (por exemplo, validar um IP, extrair um *hash* ou testar uma porta de rede). A declaração em Python inicia-se com a palavra reservada `def`, contudo, a construção de ferramentas fiáveis exige a implementação de duas metodologias de rigor: **Type Hinting** e **Docstrings**.

#### Type Hinting (Anotações de Tipo)

A tipagem dinâmica do Python permite que uma função desenhada para processar uma *string* receba dados imprevistos, como uma lista, originando a interrupção abrupta da execução. A utilização de anotações de tipo (`: str`, `: int`, `-> bool`) declara explicitamente os dados aceites e o retorno gerado. Esta prática documenta o código passivamente e permite que analisadores estáticos identifiquem potenciais injeções de dados inválidos antes do período de execução.

#### Docstrings (Documentação Interna)

O código desenvolvido para equipas de segurança deve ser rigorosamente autoexplicativo. A documentação interna é inserida na primeira linha do corpo da função, delimitada por três aspas duplas (`"""`), detalhando a finalidade técnica, os argumentos exigidos e o resultado esperado.

```
def validate_ipv4_format(ip_address: str) -> bool:
    """
    Analisa a estrutura de uma string para garantir que obedece ao formato IPv4.

    Args:
        ip_address (str): A cadeia de caracteres a ser validada.

    Returns:
        bool: True se o formato for estruturalmente válido, False caso contrário.
    """
    # Lógica de validação entraria aqui
    return(True)
```

## 1.2. Parâmetros Imutáveis vs. Mutáveis

O comportamento do interpretador Python na transferência de dados para o interior de uma função varia estritamente consoante a tipologia do dado:

### Tipos Imutáveis (Inteiros, Strings, Tuplos)

A alteração de um valor imutável dentro de uma função resulta na criação automática de uma cópia em memória local. A variável original, alojada fora da função, permanece inalterada e segura.

### Tipos Mutáveis (Listas, Dicionários, Sets)

A passagem de dados mutáveis não gera cópias de segurança na memória. A função acede diretamente à referência original. Qualquer modificação executada (como a eliminação de um IP numa lista de acessos) será refletida globalmente. Em contextos de auditoria ou forense, a alteração accidental de estruturas críticas num escopo secundário compromete a integridade de todo o processo.

## 1.3. Parâmetros por Defeito

A definição de valores por defeito confere flexibilidade às ferramentas. Num *scanner* de rede, a porta alvo pode assumir automaticamente o valor 80 caso o utilizador omita essa informação (`def scan_target(ip: str, target_port: int = 80)`).

A utilização de estruturas de dados mutáveis (como listas [] ou dicionários {}) como valores por defeito constitui um erro arquitetural.

O interpretador Python avalia e aloca em memória os valores por defeito apenas uma vez, durante a leitura inicial da função. Consequentemente, se a função adicionar elementos a essa lista em múltiplas invocações, a mesma estrutura de memória será partilhada. Este comportamento provoca a contaminação cruzada de dados, unindo registos de sessões independentes.

As boas práticas exigem a utilização de *None* como valor por defeito, inicializando a estrutura de forma isolada no interior da função.

```
from typing import Optional
def log_threat(target_ip: str = 'Teste', active_blocklist: Optional[list] = None) -> list:
    """
    Regista um IP numa lista de bloqueio de forma segura.

    Args:
        target_ip (str): O IP a registar.
        active_blocklist (Optional[list]): A lista atual. Se omitida, cria uma nova.

    Returns:
        list: A lista atualizada.
    """
    if active_blocklist is None:
        active_blocklist = []

    active_blocklist.append(target_ip)
    return active_blocklist
```

#### 1.4. Variáveis Globais vs Locais

As funções operam através de um scope Local. As variáveis declaradas internamente são alocadas em memória no momento da invocação e eliminadas pelo sistema assim que a execução atinge a instrução `return`.

O acesso forçado ao scope Global através da palavra reservada `global` é avaliado como uma má prática de engenharia.

- A utilização de `global` introduz efeitos colaterais. Numa ferramenta extensa, a modificação de variáveis fora do fluxo de execução quebra a previsibilidade lógica e impossibilita o rastreio eficaz de vulnerabilidades ou *bugs*.
- A regra arquitetural estabelece que o fluxo de informação deve ser totalmente auditável: os dados dão entrada exclusivamente através de parâmetros e o resultado do processamento é devolvido, de forma estrita, pela instrução `return`.

#### 1.5. Isolamento de Input

O interpretador Python executa instruções localizadas na raiz do ficheiro (escopo global) no exato momento da leitura. Na engenharia de software de segurança, a criação de ferramentas reutilizáveis exige que o código fonte possa ser importado por outros sistemas (por exemplo, uma ferramenta central que importa módulos de exploração ou scanners específicos). Se o ficheiro contiver rotinas de execução na sua raiz, a simples importação do módulo despoletará a execução imediata e não autorizada do ataque ou da auditoria.

O isolamento do ponto de arranque é garantido através da variável interna `__name__`. Quando um ficheiro é invocado diretamente através do terminal, o interpretador atribui a esta variável o valor `"__main__"`. Quando o ficheiro é importado por outro script, a variável assume o nome original do ficheiro, ignorando o bloco condicional.

```
# Apenas executa se invocado diretamente pelo sistema operativo
if __name__ == "__main__":
    pass
```

### 1.6. Função main

As boas práticas proíbem a colocação de lógica estrutural diretamente abaixo do bloco `if __name__ == "__main__":`. A prática standard dita que este bloco deve ter apenas uma responsabilidade: invocar a função principal de orquestração, universalmente designada por `main()`.

A função `main()` atua como o controlador de tráfego. A sua função não é executar validações pesadas ou cálculos matemáticos, mas sim ditar a sequência de execução das funções puras previamente construídas.

Também representa a fronteira entre o ambiente externo (Sistema Operativo / Linha de Comandos) e o ambiente interno (Arquitetura Funcional do *script*).

Numa ferramenta operacional, os dados voláteis (como o IP alvo, a porta ou o caminho para uma *wordlist*) não devem estar codificados *hardcoded*. A recolha destes parâmetros externos é efetuada no `main()`, frequentemente através da captura de argumentos do sistema operativo utilizando a biblioteca `sys` e a lista `sys.argv`.

Após a recolha e conversão dos dados externos, o `main()` injeta essas variáveis, através dos parâmetros das funções. Este isolamento garante que o código lógico permanece correto.

```
def run_port_scan(target_ip: str, target_port: int) -> None:
    """
    Executa a auditoria de rede a um alvo específico.
    """
    print(f"A iniciar auditoria no alvo {target_ip}:{target_port}")
    # Lógica de scan entraria aqui

def main() -> None:
    """
    Função main
    """
    # Validação do número de argumentos passados via CLI (nome_script.py + 2 argumentos)
    if len(sys.argv) != 3:
        print("Uso correto: python scanner.py <IP_ALVO> <PORTA>")
        return

    # 1. Recolha de contexto externo
    external_ip = sys.argv[1]
    external_port_str = sys.argv[2]

    # 2. Conversão e preparação
    target_port = int(external_port_str)

    # 3. Execução da função
    run_port_scan(external_ip, target_port)

if __name__ == "__main__":
    main()
```

## 1.7. Controlo de Exceções

A execução de uma ferramenta de cibersegurança não deve ser interrompida de forma descontrolada perante anomalias externas (por exemplo, a indisponibilidade de um servidor alvo ou a eliminação de um ficheiro de *logs*). A interrupção nativa do interpretador Python perante um erro gera um *Stack Trace* — um relatório de execução detalhado da falha.

Num ambiente operacional, a emissão de um *Stack Trace* para o ecrã constitui uma vulnerabilidade. Este relatório expõe caminhos absolutos do sistema de ficheiros (ex: `/opt/scripts/scanner.py`), versões de bibliotecas instaladas e fragmentos do código fonte. A implementação de blocos `try-except` permite intercetar a anomalia do sistema e substituí-la por uma mensagem de erro controlada e registada de forma segura, sem comprometer a infraestrutura subjacente.

A utilização de uma cláusula `except` genérica (`catch-all`) é errada. Esta prática oculta *bugs* estruturais do código (como divisões por zero ou chamadas a variáveis não declaradas). O tratamento de erros exige a antecipação do erro esperado.

```
try:
    # Lógica
    pass
except FileNotFoundError:
    print("Erro: O ficheiro de dicionário não foi localizado no caminho especificado.")
except PermissionError:
    print("Erro: Privilégios insuficientes para execução da leitura.")
```

Um erro arquitetural frequente consiste na dispersão de blocos try-except por todas as funções secundárias de um *script*. Funções de baixo nível (como a extração de um *hash* ou a conversão matemática de um dado) não detêm contexto abrangente para decidir se o programa deve ser encerrado ou se a anomalia pode ser ignorada.

A metodologia correta baseia-se na **Propagação de Erros (Exception Bubbling)**. As funções utilitárias devem falhar de forma limpa, permitindo que a exceção seja transferida hierarquicamente até à função orquestradora *main()*. É no bloco *main()* que se localiza o try-except central, aglomerando a decisão estrutural de abortar a ferramenta ou reportar a falha e prosseguir para a iteração seguinte.

## 1.8. Gestão de Recursos e Código de Saída

### Bloco *finally*

Independentemente do sucesso na execução ou da ocorrência de uma falha crítica, determinados recursos do sistema operativo exigem libertação estrita. A abertura de ligações de rede (sockets) ou a manipulação de ficheiros consomem file descriptors. O bloco *finally* garante a execução incondicional do código de encerramento, mitigando a corrupção de dados na memória ou o esgotamento de recursos do servidor.

### Encerramento de Processos (*sys.exit*)

Perante um erro irrecoverável (ex: a ausência de um ficheiro de configurações para o arranque da ferramenta), o processo deve abortar imediatamente. A interrupção é executada através da função *exit()* da biblioteca nativa *sys*.

Sistemas de automação (como pipelines de CI/CD ou orquestradores de segurança) avaliam o estado terminal de uma ferramenta através do seu Código de Saída (Exit Code). A invocação de *sys.exit(0)* sinaliza ao sistema operativo que a execução decorreu sem anomalias. A utilização de *sys.exit(1)* informa que o processo foi abortado devido a um erro crítico, permitindo que as plataformas externas interrompam a cadeia de ações subsequentes de forma automatizada.